The Phase-7 Credential Security Protocol

Operational Handling of API Keys, Secrets, and Exchange Access

---

Status: **v1.0 — operational companion document, not part of the Constitution's frozen scope**

Date: **August 26, 2026**

Companion to: **Phase7_Engineering_Constitution_v1.0_Rev6.pdf**, Tier 1 Items 18 through 21. The Constitution states the four credential rules that must never be violated. This document states how to actually hold to them day to day.

Origin: **Written at Viktor's direction alongside Constitution Revision 6, which adopted the credential-security gap first logged as Engineering Notes Entry #1 and sharpened by the fourth external reviewer in Entry #6**

> *A leaked key is a categorically different kind of failure from a wrong prediction. A wrong prediction costs money that was already at risk and can be learned from. A leaked key can empty an account belonging to someone who trusted this engine, and no amount of subsequent care undoes it. That asymmetry is the only justification this document needs for being more careful than feels necessary.*

Contents

How This Document Works

Four credential invariants live in the Constitution as Tier 1 Items 18 through 21. They state what must never happen. They deliberately say nothing about how keys get stored, how often they rotate, or what to do at two in the morning when one turns up in a log file — because those are operational decisions that will change as tooling and exchanges change, and a document that must hold still cannot also hold that kind of detail.

The split, and why it matters

The Constitution is a register of things that must never be violated. This is an operations manual. Mixing them would damage both: the Constitution would acquire content that needs revising every time an exchange changes its key-permission UI, and the practical guidance would inherit a scope freeze that makes it impossible to improve when someone learns something. Keeping them apart means this document can be revised freely, as often as it needs to be, without touching a ratified invariant or triggering the Constitution's change-control process.

What that does *not* mean is that this document is optional. Every practice below exists to make one of the four invariants actually hold in the real world. A rule that says “credentials are never exposed” is not self-executing; it becomes true only if somebody decided where keys live, who can read that location, and what happens when the answer turns out to be wrong. Violating a practice here doesn't violate the Constitution directly — it just makes violating the Constitution overwhelmingly likely.

The four invariants this document serves

Constitution item	What it forbids	Practices below that serve it
Tier 1, Item 18 — Read-Only Market Access	The engine holding any credential with trade-execution permissions. Categorical, not a default.	Storage (§1), Verification (§7)
Tier 1, Item 19 — Withdrawal Permissions Never Enabled	Any credential the engine touches carrying withdrawal, transfer, or fund-movement rights, in any version, ever.	Storage (§1), Verification (§7)
Tier 1, Item 20 — Credentials Are Never Exposed	Keys hardcoded, committed, logged, printed in errors, or surfaced in any output.	Storage (§1), Rotation (§2), Incident Response (§5), Verification (§7)
Tier 1, Item 21 — Operator Credentials Stay With the Operator	Any path by which another operator's key reaches the project, its author, or a third party — including telemetry and support workflows.	Third-Party Credentials (§6), Verification (§7)

Practices are numbered §1 through §7 for cross-reference from audit findings. Nothing here is an invariant, so nothing here is frozen — this document is expected to improve.

§1 — Storage

Where a credential lives determines who can read it, and almost every real-world leak is a storage decision that seemed reasonable at the time. These practices are ordered roughly by how often they're the actual cause of a leak.

1.1 Credentials Come From the Environment, Never From the Code

PRACTICE

“Keys are read at runtime from environment variables or an OS keychain / dedicated secrets manager. They never appear as a literal string anywhere in the source tree.”

This is the practice that makes several others possible. If a key lives in the environment, rotating it means changing one environment value rather than editing and redeploying code — which means rotation actually happens instead of being perpetually deferred. It also means the source tree can be shared, reviewed, or published without a separate scrubbing step, and that an accidental commit of the whole repository doesn't carry a key with it. A config file is acceptable only if it sits outside the repository and is confirmed untracked; a config file inside the repository is a committed key waiting for someone to forget.

Enforces: Tier 1, Items 18, 19, and 20

1.2 Read-Only Keys, Confirmed at the Exchange

PRACTICE

“Every key the engine uses is generated with read-only permissions at the exchange, and that permission set is visually confirmed in the exchange's own interface — not assumed from how the key was requested.”

Item 18's entire value comes from the restriction being enforced by the exchange rather than by this engine's code. That value is only real if the permission was actually set. Exchanges vary in how they present permission scopes, some default to more than read-only, and a key requested as read-only is not the same thing as a key confirmed as read-only. The confirmation is the step that converts an intention into a structural guarantee, and it takes about fifteen seconds. Withdrawal permission is checked in the same pass and must be off, per Item 19 — even though a correctly-scoped read-only key already excludes it, because the two are separate settings on several exchanges and the redundancy is cheap.

Enforces: Tier 1, Items 18 and 19

1.3 The Repository Is Configured to Refuse Credentials — PRACTICE

“A .gitignore covering .env and any local credential file is in place before the first key exists, and its effectiveness is verified rather than assumed.”

Ordering matters here more than it appears to. A .gitignore added after a key has already been committed does nothing — the file is already tracked, and git will keep tracking it. Setting this up before any credential exists is the difference between a working control and a decorative one. Verification means actually confirming the file is untracked, not reading the .gitignore and believing it. Where the tooling supports it, a pre-commit hook that scans staged changes for key-shaped strings is worth adding, because it catches the case this practice can't: a key pasted somewhere nobody thought to ignore.

Enforces: Tier 1, Item 20

1.4 File Permissions Restricted to the Owner — PRACTICE

“Any local file containing a credential is readable only by the account that needs it — mode 600 or the platform equivalent.”

This is the practice that matters least on a single-user machine and most on any shared or multi-account system, which is exactly why it's worth setting once rather than deciding case by case. It also limits what a compromised low-privilege process on the same machine can read, which is the realistic threat model for a machine that browses the internet.

Enforces: Tier 1, Item 20

1.5 Credentials Never Enter a Third-Party Tool — PRACTICE

“Keys are never pasted into a chat interface, an AI assistant, a pastebin, a support form, a screen-share, or any other system that retains what it receives.”

Worth stating explicitly because it's the newest of the common leak paths and the least instinctively guarded. Pasting a config file into an assistant to ask why something isn't working is a natural debugging move, and it puts the key into a system with its own retention and logging behavior — outside the operator's control from that moment on. This applies to Claude, in this project, as much as to any other tool: when a config file needs to be discussed, the credential values are removed first. A key that has been pasted anywhere is treated as compromised and rotated under §5, regardless of how trustworthy the destination seemed.

Enforces: Tier 1, Item 20

§2 — Rotation

Rotation is replacing a working credential with a new one and retiring the old. Its value is that it bounds how long a leak nobody noticed stays useful — which is the situation worth planning for, since undetected leaks are the common case rather than the exception.

2.1 Scheduled Rotation on a Fixed Cadence — PRACTICE

“Keys are rotated on a defined schedule — ninety days is a reasonable default — whether or not anything appears to be wrong.”

The reasoning is not that keys wear out. It's that a leak discovered on day one and a leak never discovered at all look identical from the inside, and scheduled rotation is the only control that helps in the second case. It also has a practical benefit worth as much as the security one: rotation performed regularly stays a routine two-minute operation, while rotation performed for the first time during an actual incident becomes a fumbling search for the right settings page under exactly the wrong conditions.

Enforces: Tier 1, Item 20

2.2 Immediate Rotation on Any Suspicion — PRACTICE

“Any event that could plausibly have exposed a key triggers rotation immediately, without first establishing whether exposure actually occurred.”

The temptation in the moment is to investigate first and rotate only if the investigation confirms a problem — which inverts the cost asymmetry. Rotating unnecessarily costs two minutes. Not rotating when it was necessary costs an account. Because proving a negative is usually impossible anyway, waiting for certainty means waiting indefinitely. Triggering events include: a key appearing in any log, output, or screenshot; a key pasted into any third-party tool; a commit that might have included one; a machine compromise or suspected malware; unexplained API activity; and any personnel or device change affecting who can reach the storage location.

Enforces: Tier 1, Item 20

2.3 Rotation Requires No Code Change — PRACTICE

“Replacing a credential is a configuration operation, never a code edit and redeploy.”

This is §1.1 stated as a consequence, and it's included separately because it's the property that determines whether the two practices above are realistic. If rotation requires editing source, testing, and redeploying, it will be deferred under pressure — exactly when it matters most. If it requires changing one environment value and restarting, it happens. Any design that makes rotation expensive should be treated as a finding, because it will quietly convert every rotation rule above into a suggestion.

Enforces: Tier 1, Item 20

§3 — Revocation

Revocation is telling the exchange that a key is no longer valid. It differs from rotation in a way that matters under pressure: rotation replaces a key going forward, while revocation kills the old one immediately. In an incident, revocation is the action that actually stops the bleeding.

3.1 Revocation Happens at the Exchange, Not Locally — PRACTICE

“A credential is revoked by invalidating it in the exchange's own interface. Deleting a local file, unsetting an environment variable, or shutting the engine down are not revocation.”

This is the single most important distinction in this document, because the intuitive action is the useless one. Deleting the local copy removes the operator's access to the key. It does nothing whatsoever about the copy an attacker already has — that copy remains valid, and the exchange has no way of knowing it shouldn't be. Only the exchange can actually kill a credential. Under stress the instinct is to close the laptop; the correct action is to open the exchange.

Enforces: Tier 1, Items 19 and 20

3.2 The Revocation Path Is Documented Before It Is Needed — PRACTICE

“For every exchange in use, the exact path to the key-management page and the revocation control is written down somewhere reachable without the engine running.”

Incidents are discovered at inconvenient times, and exchange interfaces bury key management in different places under different names. Locating the right settings page while an incident is live wastes the minutes that matter most, and does it under the worst possible cognitive conditions. Writing the path down in advance takes a few minutes once. Reachable without the engine running matters because the incident may be that the machine the engine runs on is compromised.

Enforces: Tier 1, Item 20

§4 — IP Allowlisting

Most exchanges allow a key to be restricted so it only works from specified network addresses. Where it's available, it's the highest-value control in this document relative to the effort it takes.

4.1 Bind Keys to Specific Addresses Wherever Supported — PRACTICE

“Every key is restricted at the exchange to the network addresses the engine actually runs from, on every exchange that offers the option.”

This control changes what a leaked key is worth. A key with no address restriction is usable by anyone who obtains it, from anywhere. The same key bound to a specific address is close to worthless to a remote attacker — they hold valid credentials the exchange will refuse to accept from them. It converts a total compromise into a non-event, and it costs one configuration step at key creation. The practical friction is that a changing home IP address means occasional updates, which is mildly annoying and worth it; running from a fixed address removes even that. Where an exchange doesn't support allowlisting, that absence should be noted, because it raises the value of every other control for keys on that exchange.

Enforces: Tier 1, Items 18, 19, and 20

§5 — Incident Response

A written sequence for the moment something has gone wrong. It exists because incident response performed from first principles under stress reliably gets the order wrong — the instinct is to understand first and act second, and that instinct is exactly backwards here.

What counts as an incident

Any of the following, regardless of how likely actual exposure seems: a credential appears in a log file, terminal output, error message, or screenshot; a credential is committed to version control, even in a branch that was never pushed; a credential is pasted into any third-party system including an AI assistant; the machine holding credentials shows signs of compromise; API activity appears that the operator cannot account for; a credential is visible during a screen-share or recording; or a device with stored credentials is lost, stolen, sold, or serviced. The threshold is deliberately low. The cost of treating a non-incident as an incident is a few minutes; the cost of the reverse is unbounded.

The sequence — in this order, without exception

Step	Action	Why this position in the order
1	Revoke the affected credential at the exchange.	First, before anything else, and before understanding what happened. This is the only step that actually stops an attacker who already has the key. Every minute spent investigating first is a minute the key still works.
2	Confirm the revocation took effect.	A revocation that silently failed leaves the operator believing they're safe while they aren't — which is worse than knowing they're exposed, because it stops further action.
3	Check the account for unauthorized activity.	Only now, with the door closed. Look at order history, balances, withdrawal history, and any active sessions or additional keys the attacker may have created.
4	Generate a replacement credential and restore service.	After the account is known to be clean. Restoring service before step 3 risks rebuilding on top of a compromise that's still active.
5	Determine how the exposure happened.	The investigation belongs here, not at the start. It's the step that prevents recurrence rather than the one that limits damage, and it's the only step that benefits from being unhurried.
6	Write it up as an Engineering Notes entry.	Per Tier 3's documentation-of-decisions principle. A leak that isn't recorded teaches nothing, and the same storage mistake will be available to make again.

The ordering principle in one line: revoke first, understand later. Every instinct under stress argues for the reverse, which is precisely why the order is written down here rather than reasoned out in the moment.

§6 — Third-Party Credentials

Everything above concerns protecting Viktor's own account access. This section concerns a different and more serious obligation: what happens if anyone else ever runs this engine with their own exchange credentials. Tier 1, Item 21 makes this an invariant; these are the practices that make it hold.

6.1 No Path Exists For a Credential to Reach the Project — PRACTICE

“The engine contains no mechanism — no upload endpoint, no central key store, no registration flow, no configuration sync — capable of transmitting an operator's credentials to the project, its author, or any third party.”

This is stated as an architectural absence rather than a policy, because a policy against using a capability is weaker than not having the capability. Item 21 was deliberately written before any multi-user functionality exists, which means this practice constrains a design that hasn't been built yet rather than describing one that has. That ordering is the whole point: once a credential-collection path exists for a good reason, removing it becomes a refactor nobody has time for.

Enforces: Tier 1, Item 21

6.2 Diagnostics Cannot Carry a Credential Out — PRACTICE

“Any telemetry, crash reporting, error aggregation, or usage analytics the engine ever gains must be incapable of including credential material — verified by inspecting what is actually transmitted, not by trusting the library's defaults.”

This is the indirect path that defeats good intentions, and it's how a project that would never dream of collecting keys ends up holding them anyway. Crash reporters commonly capture environment variables and local variable state by default, which is exactly where credentials live under §1.1. The failure is entirely accidental, invisible from the outside, and discovered only by looking at the payload. If such a channel is ever added, the verification is inspecting a real transmitted report, not reading the documentation.

Enforces: Tier 1, Items 20 and 21

6.3 Support Never Asks For a Credential — PRACTICE

“No support, onboarding, or troubleshooting workflow ever asks an operator to share a key, a config file, or a screenshot containing either. If an operator volunteers one anyway, it is treated as an incident on their behalf and they are told to rotate immediately.”

The social path is the one no technical control covers. An operator with a broken setup will often offer their config file unprompted, because it feels like the helpful thing to do, and a support process that accepts it has just collected a credential regardless of what the architecture forbids. The obligation runs the other way in that moment: tell them to revoke and rotate, immediately, before troubleshooting anything. Normalizing key-sharing as a debugging step is how it becomes routine.

Enforces: Tier 1, Item 21

§7 — Verification: What the Audit Can Actually Check

Constitution Item 18 sits in the Minimum Viable Audit gate, which means it gets checked before almost anything else. This section exists so that check has something concrete to test against rather than a judgment call — and so a “Compliant” finding on any credential invariant is backed by an artifact, per Revision 3's evidence requirement.

Invariant	The check	What counts as evidence
Item 18 — Read-Only	Inspect the permission scope of every key the engine can reach, in the exchange's own interface. Confirm no trade permission is enabled.	A screenshot or exported permission listing from the exchange, with the key identifier visible and the secret not. Not a statement that the key was requested as read-only.
Item 19 — No Withdrawal	In the same pass, confirm withdrawal and transfer permissions are off, separately from the read-only scope.	The same artifact as above, showing the withdrawal permission explicitly disabled rather than merely absent from the description.
Item 20 — Never Exposed	Search the full repository history — not just the working tree — for key-shaped strings. Confirm credential files are untracked. Review a real sample of log output and at least one deliberately triggered error trace.	The search command and its output, the untracked-status output, and the actual log and trace samples reviewed. A clean working tree alone is not evidence, since history is where committed keys survive.
Item 21 — Operator Credentials	Confirm by inspection that no code path transmits credential material outward, and that no telemetry or crash-reporting channel exists — or, if one does, inspect a real transmitted payload.	The list of outbound network calls the engine makes and what each carries. If no telemetry exists, the evidence is that absence, confirmed by inspection rather than asserted.

Two notes for whoever performs this check. First, Item 20's repository search must cover history, because the failure mode being tested for is a key that was committed once and then removed — which leaves the working tree clean and the key fully recoverable. Second, these checks are all things an independent auditor can perform from the audit package without trusting anyone's account of the system, which is the property Constitution Revision 6 was written to preserve. A credential finding that rests on “Claude confirmed this is handled correctly” has failed the check regardless of whether the underlying claim is true.

Document History

Version	Date	Notes
v1.0	August 26, 2026	First version, written alongside Constitution Revision 6 at Viktor's direction. Covers storage, rotation, revocation, IP allowlisting, incident response, third-party credentials, and audit verification. Serves Tier 1 Items 18 through 21. Closes the operational half of the credential gap first logged as Engineering Notes Entry #1 and sharpened by the fourth external reviewer in Entry #6.

Unlike the Constitution, this document is not frozen and is expected to change. New exchanges, new tooling, and — most valuably — anything learned from an actual incident all belong here, as a new numbered version and a new dated row. Improving this document never requires touching a ratified invariant.